МИНОБРНАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении курсовой работы по дисциплине «Функциональное программирования» на тему «Рекомендательная система фильмов при помощи модели, основанной на косинусной мере сходства»

Студент группы 5130201/40101 _____ Резчик Д.О.

Преподаватель _____ Моторин Д.Е.

«____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

1 Введение 3

2 Описание разработки проекта 4
2.1 Архитектура программного комплекса 4
2.2 Технические особенности реализации 4
2.2.1 Загрузка данных 4
2.2.2 Предобработка текста 5
2.2.3 Разбиение датасета и построение TF-IDF 5
2.2.4 Косинусная мера сходства 5
2.2.5 Оценка качества модели 5
2.2.6 Формирование рекомендаций 5
2.2.7 Кеширование модели 6
2.2.8 Пользовательский интерфейс 6
2.2.9 Логирование и обработка ошибок 6
2.2.10 Тестирование 6

3 Примеры работы программы 7
3.1 Запуск программы 7
3.2 Рекомендации на основе ручного ввода фильмов 7
3.3 Загрузка списка фильмов из файла 8
3.4 Поиск названия фильма по составляющим 8
3.5 Свободный ввод описания фильма 9

4 Руководство по эксплуатации 9
4.1 Требования к окружению 9
4.2 Запуск 9

5 Заключение 10
5.1 Результаты тестирования 10
5.2 Диаграмма Ганта 11

6 История коммитов 11

7 Примеры использования нейронных сетей 13

Список литературы 14

1 Введение

Рекомендательные системы — программные средства, которые предлагают пользователю объекты (фильмы, товары, тексты) на основе его предпочтений и характеристик самих объектов. По способу построения рекомендаций различают коллаборативную фильтрацию, фильтрацию по содержанию (content-based) и гибридные подходы [5]. В content-based системах сходство оценивается по описанию объекта: тексту, жанрам, ключевым словам и другим метаданным.

Для представления текстов в информационном поиске широко применяется векторная модель. Веса терминов часто вычисляют по схеме TF-IDF: учитывается частота термина в документе и его специфичность относительно всей коллекции [?, 6]. Близость документов после векторизации обычно измеряют косинусной мерой сходства, которая зависит от угла между векторами и слабо чувствительна к длине описания [?, 6].

Content-based рекомендация фильмов на основе TF-IDF является базовым и хорошо интерпретируемым подходом: похожие картины ищутся по пересечению тематики сюжета, жанров и ключевых слов [5, 7]. Такой метод не требует большой матрицы пользовательских оценок.

Реализация пайплайна на языке Haskell соответствует принципам функционального программирования: данные неизменяемы, преобразования задаются чистыми функциями, а операции ввода-вывода выносятся на границу программы [1, 3]. Для проверки свойств функций применяют property-based тестирование; классическим инструментом в экосистеме Haskell является QuickCheck [8].

В данной работе предметная область ограничена content-based рекомендацией фильмов по текстовым признакам. Источником данных служит открытый набор Movie Dataset [9].

2 Описание разработки проекта

2.1 Архитектура программного комплекса

Проект реализован на языке Haskell использованием системы сборки Stack версии 3.9.1 и GHC версии 9.6.7. Исходный код разделён на следующие модули:

- Types — общие типы предметной области: Movie, ProcessedMovie, SparseVector, TFIDFModel;
- CSVLoader — чтение CSV-файла с данными о фильмах и преобразование записей в структуру Movie;
- Preprocessing — предобработка текста: приведение к нижнему регистру, удаление пунктуации, токенизация, фильтрация стоп-слов и коротких токенов, разбор жанров и ключевых слов
- TFIDFVectorizer — разбиение датасета на train/val/test, построение словаря и IDF по обучающей выборке, вычисление sparse TF-IDF векторов, косинусная мера сходства;
- ModelEvaluation — расчёт метрик качества модели: покрытие датасета, средняя разреженность векторов, среднее сходство фильмов жанра с центроидом жанра;
- ModelPersistence — сохранение и загрузка обученной модели (словарь, IDF-веса, векторы фильмов) в CSV-файлы кеша;
- Recommender — построение среднего вектора выбранных фильмов и формирование топ-N рекомендаций по косинусной мере сходства;
- UI — консольное меню: ручной ввод названий фильмов, загрузка списка из файла, поиск по названию, вывод рекомендаций;
- Logger — запись событий работы программы в журнал app.log с временными метками.;
- Main — точка входа: загрузка данных, разбиение датасета, построение или загрузка модели, запуск пользовательского интерфейса.

2.2 Технические особенности реализации

2.2.1 Загрузка данных

Модуль CSVLoader читает CSV-файл с помощью библиотеки cassava и функции decodeByName. Для каждой строки формируется запись типа Movie: извлекаются идентификатор, оригинальное название, описание сюжета, ключевые слова, жанры и показатель популярности. Соответствие колонок полям задаётся экземпляром FromNamedRecord. Если файл отсутствует или строка не может быть разобрана, функция возвращает ошибку в виде Left с текстовым сообщением, которое затем обрабатывается в Main и фиксируется в журнале.

2.2.2 Предобработка текста

Модуль Preprocessing преобразует Movie в ProcessedMovie. Описание фильма приводится к нижнему регистру, знаки пунктуации заменяются пробелами, выполняется разбиение на токены. Затем удаляются стоп-слова и токены длины меньше трёх символов. Жанры и ключевые слова, представленные в датасете строками через пробел, разбираются аналогичным образом. Итоговый набор токенов фильма формируется как объединение токенов overview, genres и keywords; жанровые токены дополнительно усиливаются повторением, чтобы повысить их вклад в векторное представление.

2.2.3 Разбиение датасета и построение TF-IDF

Модуль TFIDFVectorizer выполняет разбиение предобработанных фильмов на обучающую, валидационную и тестовую выборки в пропорции 70/15/15. Порядок элементов задаётся детерминированно, чтобы результаты воспроизводились между запусками. На обучающей выборке строится словарь уникальных термов и вычисляются значения IDF с аддитивным сглаживанием. Для каждого фильма формируется sparse TF-IDF вектор типа Map Int Double, в котором хранятся только ненулевые компоненты. Это позволяет компактно представлять высокоразмерные описания без плотной матрицы.

2.2.4 Косинусная мера сходства

Функция cosineSimilarity принимает два sparse-вектора и вычисляет отношение их скалярного произведения к произведению евклидовых норм. Скалярное произведение считается только по общим ключам. Если хотя бы один вектор имеет нулевую норму, результат полагается равным нулю. Полученная величина лежит в диапазоне от 0 до 1 для неотрицательных TF-IDF весов и используется как оценка близости фильмов.

2.2.5 Оценка качества модели

Модуль ModelEvaluation рассчитывает три показателя на валидационной выборке. Покрытие определяется как доля фильмов, для которых успешно построен вектор. Разреженность вычисляется как средняя доля ненулевых компонент вектора относительно размера словаря. Для оценки смысловой согласованности внутри жанра строится центроид TF-IDF векторов фильмов данного жанра, после чего усредняется косинусное сходство каждого фильма жанра с этим центроидом. Итоговые метрики выводятся при построении модели и записываются в лог.

2.2.6 Формирование рекомендаций

Модуль Recommender получает идентификаторы выбранных пользователем фильмов и извлекает соответствующие TF-IDF векторы. По ним строится сред-

ний вектор профиля. Для каждого оставшегося фильма вычисляется косинусная близость к профилю. Кандидаты сортируются по убыванию сходства, после чего возвращается заданное число лучших результатов вместе с названием и числовой оценкой. В пользовательском интерфейсе оценка отображается в процентах.

2.2.7 Кеширование модели

Модуль ModelPersistence сохраняет обученную модель в три CSV-файла: словарь термов, таблицу IDF и sparse-векторы фильмов. При запуске приложения Main проверяет наличие каталога кеша. Если файлы найдены, модель загружается без повторного пересчёта TF-IDF; иначе выполняется полное построение, после чего результат записывается на диск. Такой подход сокращает время повторных запусков.

2.2.8 Пользовательский интерфейс

Модуль UI реализует консольное меню с четырьмя режимами: ручной ввод названий фильмов через запятую, загрузка списка названий из текстового файла, поиск фильма по подстроке названия и завершение работы. Поиск выполняется без учёта регистра. Для каждого введённого названия система сообщает, найден фильм или нет. После успешного выбора вызывается Recommender, и пользователю выводится топ-10 рекомендаций.

2.2.9 Логирование и обработка ошибок

Модуль Logger записывает события в файл app.log с меткой времени и уровнем сообщения (INFO, WARN, ERROR). Логируются загрузка данных, построение и загрузка модели, выбор пунктов меню, результаты поиска фильмов, запросы рекомендаций и ошибки. Отсутствие CSV-файла, повреждённый кеш и некорректный ввод пользователя обрабатываются без аварийного завершения: программа выводит понятное сообщение и продолжает работу либо корректно завершает текущий сценарий.

2.2.10 Тестирование

Для проверки корректности ключевых чистых функций используется библиотека QuickCheck. Тесты расположены в модуле test/Spec.hs и проверяют свойства, которые должны выполняться для произвольных входных данных. Для предобработки текста проверяется, что после preprocessText все токены записаны в нижнем регистре, не содержат пустых строк, имеют длину не менее трёх символов и не входят в множество стоп-слов. Для косинусной меры сходства проверяются симметричность, принадлежность результата допустимому числовому диапазону, нулевое сходство с пустым вектором и равенство единице при сравнении ненулевого вектора с самим собой. Для рекомендаций проверяется, что выбранные

пользователем фильмы не попадают в список результатов и что число выданных рекомендаций не превышает заданный параметр topN.

Генераторы тестовых данных формируют произвольные строки, sparse-векторы с неотрицательными весами и списки ProcessedMovie. Запуск тестов выполняется командой stack test. Все описанные свойства успешно проходят на наборах из 100 случайных примеров.

3 Примеры работы программы

3.1 Запуск программы

После запуска проекта, пользователю выводится меню программы(рис.1). В меню доступны следующие действия: ввод названия фильмов; загрузка списка фильмов из txt файла; найти полное название фильма по части названия; поле свободного ввода описания фильма; выход из программы

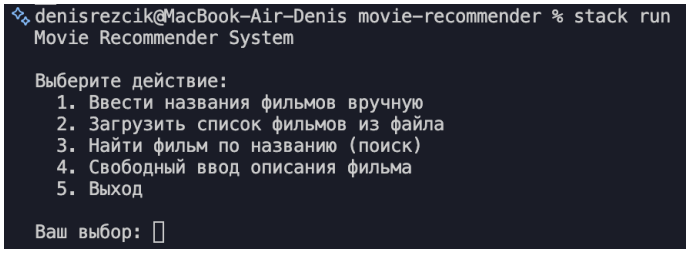


Рис. 1. Главное меню программы

3.2 Рекомендации на основе ручного ввода фильмов

При выборе первого пункта меню пользователь сможет самостоятельно вводить название интересных ему фильмов в консоль через запятую. В результате будут выведены 10 фильмов, наиболее общих для перечисленных пользователем. Пример рекомендаций представлен на рис.2.

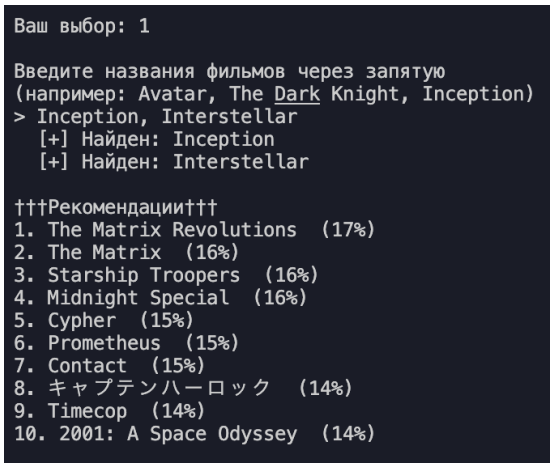


Рис. 2. Рекомендации на основе ручного ввода

3.3 Загрузка списка фильмов из файла

При выборе второго пункта, пользователь может загрузить txt файл с названиями фильмов, рекомендации к которым желает получить пользователь. Пример загрузки и рекомендаций представлен на рис.3.

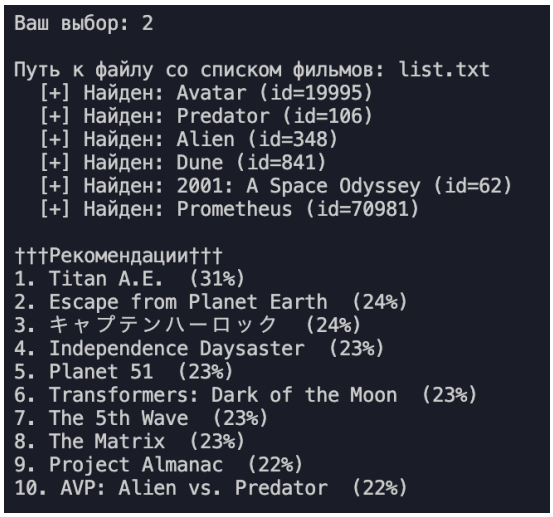


Рис. 3. Пример загрузки фильмов и последующих рекомендаций

3.4 Поиск названия фильма по составляющим

При выборе третьего пункта пользователь может ввести часть названия фильма в любом формате. В результате будут выведены фильмы, в названиях которых есть данное слово/часть слова. Пример поиска представлен на рис.4.

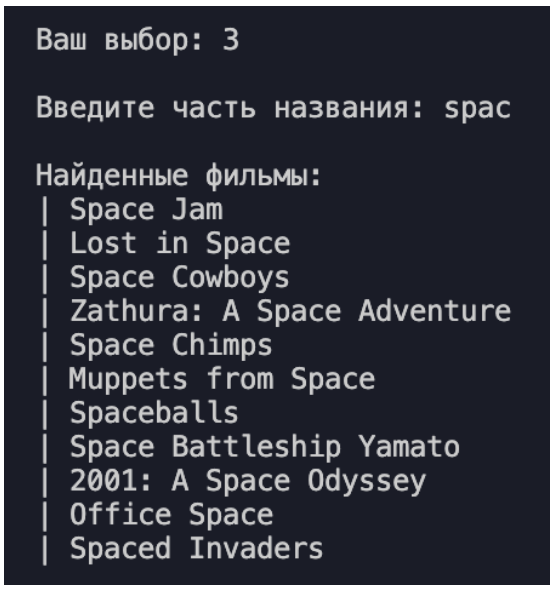


Рис. 4. Пример поиска

3.5 Свободный ввод описания фильма

Если фильм не был найден в базе данных, пользователь может выбрать четвёртый пункт меню и собственноручно ввести описание фильма. Пример ввода и последующих рекомендаций представлен на рис.5



Рис. 5. Ручной ввод описания фильма

4 Руководство по эксплуатации

4.1 Требования к окружению

- GHC версии 9.6 и новее;
- Stack версии 3 и новее;
- файл датасета movies.csv;
- зависимости проекта из package.yaml.

4.2 Запуск

```
1 stack build
2 stack run
3 stack test
```

stack build — команда утилиты Stack, которая автоматически собирает проект на языке Haskell.

stack run— команда, собирающая исполняемый файл проекта и запускающая его.

stack test — команда, которая автоматически собирает проект и запускает все прописанные в нём тесты.

После запуска проекта с помощью команды stack run пользователю в консоль выведется меню, дальнейшая работа в котором и будет являться результатом выполнения программы:

1. Ввести названия фильмов вручную (через запятую);

2. Загрузить список фильмов из текстового файла (по одному названию на строку);
3. Найти фильм по части названия;
4. Свободный ввод описания фильмов;
5. Выход.

5 Заключение

В ходе выполнения курсовой работы был разработан програмнный комплекс для рекомендаций фильмов с использованием косинусной меры сходства. Программа реализована на языке Haskell и запускается с помощью Stack.

В проекте реализованы чтение и парсинг CSV-файлов, предобработка текстовых описаний, построение TF-IDF векторов, разбиение датасета на обучающую, валидационную и тестовую выборки, оценка качества модели, кеширование результатов, консольный интерфейс, логирование и набор тестов QuickCheck. Рекомендации формируются как по выбранным фильмам из датасета, так и по произвольному текстовому описанию.

Полученные на валидационной выборке метрики показывают полное покрытие данных и среднее сходство фильмов внутри жанра около 0.29, что близко к заданному в техническом задании порогу. Система позволяет пользователю искать фильмы, загружать список названий из файла и получать топ-10 наиболее близких по содержанию рекомендаций.

В дальнейшем проект может быть улучшен возможностью добавления новых фильмов, интеграцией полноценной базы данных для хранения фильмов.

5.1 Результаты тестирования

Для проверки корректности отдельных частей программы используются тесты свойств QuickCheck. Проверяются следующие свойства:

- после предобработки текста все токены записаны в нижнем регистре;
- в результате предобработки отсутствуют пустые токены;
- длина каждого токена не меньше трёх символов;
- стоп-слова не входят в итоговый список токенов;
- косинусная мера сходства вектора с самим собой равна 1 для ненулевого вектора;
- косинусная мера симметрична относительно своих аргументов;
- значение косинусной меры лежит в диапазоне от 0 до 1 для неотрицательных TF-IDF векторов;
- сходство с нулевым вектором равно 0;
- выбранные пользователем фильмы не появляются в списке рекомендаций;
- число возвращаемых рекомендаций не превышает заданный параметр topN.

Тесты запускаются командой stack test. Все указанные свойства успешно выполняются на наборах из 100 случайно сгенерированных примеров.

5.2 Диаграмма Ганта

На рис.6 представлено сравнение запланированных и фактических сроков выполнения этапов. Запланированные сроки выделены синим цветом, фактические красным.



Рис. 6. Сравнение запланированной и фактической диаграммы Ганта

6 История коммитов

Коммит 1d213f0

Время коммита: 08.09.2026, 16:15
Описание: создание нового технического задания для курсовой работы

Коммит 652fb67

Время коммита: 10.09.2026, 00:14
Описание: Реализовал модули CSVLoader, Preprocessing, Types. Отвечают за загрузку фильмов из файла; нормализацию текста, токе низацию, удаление стоп-слов и пунктуации; описывают типы данных для фильмов. Реализованы парсинг CSV файла по нужным по лям, предобработка описаний фильмов и успешная загрузка и обработка 4803 фильмов

Коммит 87d5676

Время коммита: 10.09.2026, 13:24
Описание: Изменение модуля Types: добавлены типы SparseVector и TFIDFModel. Изменение модуля Preprocessing: исправил парсинг genres и keywords, добавлена

фильтрация коротких токенов(длина <3), жанры повторяются для усиления влияния на сходство. Реализован модуль TFIDFVectorizer – построение словаря, расчёт IDF, создание sparse TF-IDF векторов, косинусная мера с ходства. Реализован модуль ModelEvaluation – вычисление метрик качества модели. Итоговое среднее сходство в жанре прибл изительно равно 0,223

Коммит 710d515

Время коммита: 11.09.2026, 22:13

Описание: Реализовал модули для работы и кэширования данных модели, а также консольного приложения. Recommender: строит средн ий вектор выбранных фильмов, ранжирует кандидатов по косинусному сходству, выдаёт топ-10 фильмов с указанием степени сх одства. ModelPersistence: сохраняет словарь, IDF веса и sparse-vector в CSV. UI: интерактивное меню, ввод фильмов вручн ую, загрузка фильмов из текстового файла, поиск фильма по части названия и обработка случаев, когда фильм не введён.

Коммит 09ed078

Время коммита: 13.09.2026, 00:10

Описание: Добавление функций:

- splitDataset – разбиение датасета на train/val/test (70/15/15)
- buildTFIDFModelFromSplit – построение TF-IDF модели: словарь и IDF только по train, векторы для всех выборок
- evaluateModel (доработана) – расчёт метрик только по переданной выборке (validation)

Рефакторинг:

- ModelEvaluation – исправлен расчёт coverage (учитываются только векторы оцениваемых фильмов)
- TFIDFVectorizer – разделена логика обучения (train) и применения модели

Исправление частичных несоответствий ТЗ:

- словарь и IDF строятся только на обучающей выборке
- метрики качества считаются на валидационной выборке
- покрытие датасета корректно отображается (100

Добавление / доработка:

- Logger – логирование всех действий UI (меню, поиск, рекомендации, ошибки)
- QuickCheck-тесты: preprocessText, cosineSimilarity, recommend.

также были исправлены несколько опечаток в ТЗ.

Коммит af0be08

Время коммита: 13.09.2026, 18:05

Описание: Добавление диаграммы типов и модулей

Коммит 816cb97

Время коммита: 14.09.2026, 23:04

Описание: Рефакторинг: Logger больше не выводит логи программы в консоль во время работы программы. Новый функционал: добавил возможность ввода описания фильма в свободной форме(функции recommendByText, vectorizeText, recommendFromFreeText).

7 Примеры использования нейронных сетей

Приведённые ниже задачи решались при помощи нейронной сети Grok.

задача	промпт	результат
Парсинг файла movies.csv	Как мне парсить данные из CSV файла с помощью библиотеки cassava на языке Haskell? Приведи примеры, и расскажи о функционале	Инструкция и примеры использования библиотеки cassava
Перечисление стоп-слов для векторизации	Есть вопрос по TF-IDF векторизации: понятно, что сперва нужно перечислить стоп-слова(артикли всякие, предлоги и т.д). можешь заполнить полностью все возможные стоп-слова?	Функция englishStopWords с перечисленными стоп-словами
Повышение качества работы косинусной меры сходства с помощью центроида	Смотри, я вычитал кое-какую информацию о том, что можно максимизировать косинусную меру сходства, если использовать центроид. Расскажи мне об этом поподробнее и приведи примеры использования. Покажи наглядный пример построения центроида	Улучшенная модель с использованием центроида
Написание тестов QuickCheck	этой мой текущий полный проект. Нужно написать для него тесты с использованием QuickCheck. Какие части проекта можно тестировать?	Можно тестировать модули предобработку текста, косинусную меру, центроид и TF-IDF.

Таблица 1. Документирование использования нейронных сетей

Список литературы

[1] Душкин Р. В. Функциональное программирование на языке Haskell. М.: ДМК Пресс, 2006. — 608 с.

[2] Извлечение признаков из текстовых данных с использованием TF-IDF // Электронный ресурс // URL: https://habr.com/ru/companies/otus/articles/755772/ (дата обращения: 10.09.2026)

[3] Кубенский А. А. Функциональное программирование: учебно-методическое пособие. СПб.: СПбГУ ИТМО, 2010. — 251 с.

[4] Что такое косинусная близость? // Электронный ресурс // URL: https://sysblok.ru/glossary/chto-takoe-kosinusnaja-blizost/

[5] Рекомендательная система // Электронный ресурс // URL: https://ru.wikipedia.org/wiki/РґРҳРэР«РёРҳР„РұРөСЂРҳРњСЃР„РөCS_СЃРчСЃСЂРҳРёРө (дата обращения: 14.09.2026).

[6] Векторная модель // Электронный ресурс // URL: https://ru.wikipedia.org/wiki/РЎРҳРэСЂР«СЃР„РөCS_РёР«РұРҳРњСЃ (дата обращения: 14.09.2026).

[7] Коновалов А. Как TF-IDF обошел SOTA-модель BERT4Rec в персональных рекомендациях. URL: https://pcnews.ru/blogs/kak_tf_idf_obosel_sota_model_bert4rec_v_personalnyh_rekomendaciah-1559682.html (дата обращения: 14.09.2026).

[8] Claessen K., Hughes J. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs // Proc. of ICFP. 2000. P. 268–279.

[9] Movie Dataset [Электронный ресурс]. URL: https://www.kaggle.com/datasets/figolm10/movie-dataset (дата обращения: 25.08.2026).